TecnoStock · Unidad 2 · Diseño Arquitectónico del Sistema

1. Entregable 01 · Elección del patrón de lógica de negocio

1.1 Tabla comparativa (patrones de Fowler, PoEAA)

Escala: ● cumple bien · ◐ cumple parcialmente · ○ cumple mal.

Criterio (origen en TecnoStock)	Transaction Script	Table Module	Active Record	**Domain Model**
Cantidad y complejidad de reglas (12 reglas implícitas, estados Activo/Completado/Cancelado/Vencido)	○ if/else dispersos	◐	◐	● reglas dentro de los objetos
Invariantes críticas: serial único (N1), cero dobles ventas (N2), anticipo ≥ 25%, vencimiento = creación + 14 días	○ se repiten en cada script	◐	◐	● protegidas por Aggregates
Política de devolución (30 días, comprobante O supervisor, > $500, daño) y su evolución	○ crece sin control	◐	◐	● Domain Service / Specification
Ciclos de vida con estados (Apartado, NumeroDeSerie, Devolución)	○	○	◐	● máquinas de estado en la entidad
Trazabilidad y auditoría de todo cambio (regla 9)	◐	◐	◐	● Domain Events

Criterio (origen en TecnoStock)	Transaction Script	Table Module	Active Record	**Domain Model**
Testabilidad sin base de datos (criterios Gherkin verificables)	◐	○	○ acoplado a BD	● dominio puro
Integración con el PdV Windows actual (restricción BRD)	● simple	◐	◐	● vía Service Layer (adaptador)
Costo inicial / curva de aprendizaje	● bajo	● bajo	● bajo	◐ mayor
Reportes simples (US17 stock por sucursal)	● ideal	●	●	◐ innecesario

Justificación Arquitectónica Detallada: Selección del Patrón de Lógica de Negocio

Empresa Asignada: TecnoStock (Retail de Electrónica y Componentes)

1. Planteamiento de la Decisión y Contexto Operativo

Para el desarrollo del núcleo transaccional del sistema de TecnoStock —una cadena de cinco sucursales dedicada a la comercialización de computadoras, celulares, accesorios y componentes—, el equipo de arquitectura ha seleccionado la combinación del patrón Domain Model (*Modelo de Dominio*) de Martin Fowler, coordinado a través de una Service Layer (*Capa de Servicios*).

Asimismo, se establece una estrategia pragmática y acotada de lectura: para aquellas operaciones puramente analíticas o de consulta que no alteran el estado ni aplican reglas de negocio (tales como la consulta de disponibilidad en tiempo real o el reporte general de stock por sucursal correspondientes a las historias US01, US15 y US17), se permiten accesos directos de lectura optimizada bajo un enfoque tipo *Transaction Script / Read Model*, evitando sobrecargar el modelo de dominio con consultas pesadas que no protegen invariantes.

La elección de *Domain Model* no responde a una tendencia tecnológica ni a criterios genéricos de "modernidad", sino a la necesidad imperativa de resolver los fallos estructurales y operativos detectados en la evaluación del estado actual (*AS-IS*) de la compañía.

2. Análisis de los Problemas Concretos del Caso que Justifican el Patrón

El análisis de requerimientos de negocio (BRD) de TecnoStock evidenció que los desafíos prioritarios (necesidades N1, N2, N3, N5 y N7) no consisten en una simple captura, almacenamiento o despliegue tabular de datos, sino en la vulnerabilidad de las invariantes de negocio y políticas comerciales complejas. En la operación actual, la ausencia de un núcleo de dominio centralizado provoca consecuencias financieras directas que el nuevo software debe erradicar por completo:

- Venta accidental de productos apartados (Invariante N2): Actualmente, un producto reservado por un cliente mediante un anticipo mínimo del 25% puede ser vendido por error a otro comprador en el punto de venta debido a que la fórmula de disponibilidad no se evalúa de manera transaccional.

Domain Model resuelve este problema al concentrar la regla dentro del agregado StockSucursal, cuya fórmula exclusiva (disponible = total – reservado – dañado – en tránsito) rechaza de forma automática cualquier intento de venta si la mercancía se encuentra bloqueada bajo el estado RESERVADO.

- Dificultad en el control y unicidad de números de serie (Invariante N1): En la venta de equipos electrónicos de alto valor (laptops, smartphones, tablets), el sistema actual carece de un mecanismo estricto que garantice la unicidad global del número de serie. Con *Domain Model*, cada dispositivo se modela como un agregado independiente (NumeroDeSerie) que administra su propio ciclo de vida e historial inmutable, impidiendo que un mismo serial sea asignado a dos transacciones simultáneas.
- Inconsistencia en las políticas de devolución (Invariante N5): El personal de caja aplica criterios subjetivos y dispares entre las cinco sucursales al procesar devoluciones. El modelo propuesto centraliza la validación de plazos (30 días), la obligatoriedad del comprobante o la necesidad de autorización del supervisor en un *Domain Service* y objetos de especificación, garantizando un comportamiento totalmente estandarizado y auditable en toda la cadena.

3. Por qué los Demás Patrones de Fowler Resultan Insuficientes

Para fundamentar con rigor la decisión, se contrastó el comportamiento de TecnoStock frente a las alternativas tradicionales del catálogo de patrones de arquitectura empresarial (*PoEAA*):

1. Transaction Script (Descartado): Organiza la lógica mediante procedimientos lineales o scripts por cada transacción. Aunque su costo inicial es bajo, en TecnoStock propiciaría la dispersión de condicionales if/else a lo largo de las pantallas del PdV Windows y scripts de base de datos. Las 12 reglas implícitas del negocio terminarían duplicándose, lo que generaría discrepancias operativas inevitables al modificar una política comercial.
2. Table Module (Descartado): Agrupa la lógica alrededor de una sola clase por tabla de base de datos para manejar conjuntos de registros. Este patrón resulta inoperante al modelar comportamientos ricos y máquinas de estado complejas, tales como la transición de un Apartado (Activo, Completado, Cancelado, Vencido) o el ciclo de vida de un componente en inventario, acoplando indeseablemente la aplicación al esquema relacional de las tablas.

3. Active Record (Descartado): Fusiona los datos de la fila con la lógica de negocio básica en una misma clase. Si bien es útil para operaciones CRUD sencillas, colapsa cuando las entidades deben proteger invariantes cruzadas entre múltiples conceptos antes de persistir los cambios, filtrando detalles de infraestructura hacia la lógica de la aplicación.

4. Costos, Consecuencias y Contrapartidas (*Trade-offs*) Introducidas

La adopción de *Domain Model + Service Layer* no está exenta de costos técnicos; el equipo de arquitectura asume consciente y explícitamente las siguientes contrapartidas:

- Curva de aprendizaje elevada y mayor complejidad inicial: El desarrollo exige un esfuerzo de abstracción riguroso para modelar los límites de consistencia (*Bounded Contexts*), agregados y objetos de valor antes de escribir código operativo, lo que incrementa el tiempo de diseño previo.
- Incremento de clases e indirección: Se introduce una separación estricta mediante el patrón *Repository* y mapeadores de objetos, lo que eleva notablemente el volumen inicial de clases en comparación con soluciones procedurales.
- Gestión de concurrencia: Al operar simultáneamente en cinco sucursales y canales remotos, el modelo requiere implementar un control de concurrencia optimista (con números de versión) en los agregados de stock para evitar bloqueos pesimistas lentos, obligando a la *Service Layer* a gestionar reintentos controlados en caso de colisiones de inventario.

5. Conclusión y Formalización

En conclusión, la elección de Domain Model + Service Layer para TecnoStock se encuentra plenamente justificada porque es el único enfoque arquitectónico capaz de proteger las invariantes críticas del negocio, unificar los criterios de operación de las sucursales y permitir una integración limpia y no intrusiva con el punto de venta Windows existente a través de adaptadores en la capa de servicios. Esta decisión queda formalmente registrada en el documento de arquitectura bajo el identificador ADR-001.